A. Introduction

For this project, I implemented a custom 10×10 Gridworld where an agent learns to reach a goal using reinforcement learning. The environment is dynamic and includes walls (that trap

movement), traps (deceptive with negative large negative reward), slippage, and a moving goal. There's also a hard limit of 100 steps per episode.

I trained three agents on this environment:

1. **Tabular Q-Learning**
2. **DQN (Deep Q-Network)**
3. **Double DQN**

The environment, replay buffers, training loops, and evaluations are implemented manually. The neural networks were implemented using PyTorch. I ran multiple experiments by varying buffer size, slip probability, and target network sync frequency and compared them over the three agents to find the best behavior.

B. Dataset and Environment Design

I used a .csv file (grid_layout_10x10.csv) to define the base layout. Each cell is:

- S: agent's starting position (only one allowed)
- G: goal position (can be multiple)
- #: wall (agent can't move through)

- . : normal tile (safe, passable)

grid_layout_10x10

	0	1	2	3	4	5	6	7	8	9
0	.	.	.	#	.	.	.	.	.	G
1	.	#	.	#	.	#	.	#	.	.
2	.	#	S	.	.	#	.	.	#	.
3	.	.	.	.	#	.	.	.	.	.
4	#	.	#	.	.	.	#	.	.	.
5	.	.	.	#	.	.	.	#	.	.
6	.	#	.	.	#	.	.	.	#	.
7	.	.	.	.	.	#	.	.	.	.
8	#	.	.	#	.	.	.	.	#	.
9	.	.	.	.	#	.	.	.	.	.

The environment parses this file into a NumPy array and builds a list of valid states (i.e., anything that's not a wall). Each state gets mapped to an index for compatibility with Q-tables and neural nets.

```
df = pd.read_csv(csv_path, index_col=0)
self.grid = df.values  #saving as 2d array
```

```
self.states = []
for r in range(self.n_rows):
    for c in range(self.n_cols):
        if self.grid[r, c] != '#':
            self.states.append((r, c))
self.state_to_idx = {s: i for i, s in enumerate(self.states)}
```

Then, I wrapped this into a GridworldEnv class, which handles episode logic and dynamics.

Environment Features:

Slippage (default 10–20%)

- With some probability, the agent ignores its intended action and takes a random one.
- Forces the agent to learn under uncertainty and pure memorization doesn't work.

```
def step(self, action_idx):
    #slipping randomly with some probability
    if np.random.rand() < self.slip_prob:
        action_idx = np.random.randint(len(self.actions))
```

Traps

- Traps look like regular tiles (.) but apply a hidden penalty when stepped on.
- I manually passed in the coordinates as a list (e.g., (4,4), (7,2)).
- This tests whether the agent can learn to avoid unsafe states even when they look safe.

Moving Goal

- The goal changes position every few episodes (default: every 5).
- Optionally, it can move mid-episode too (goal_move_interval).
- This breaks fixed path planning. Agent has to generalize and re-plan often.

```
    if self.steps_since_goal_change >= self.goal_move_interval:
        self.goal = random.choice(self.goal_positions)
        self.steps_since_goal_change = 0
```

Timeout (default: 50 or 100 steps)

- The episode ends if the agent takes too long.
- Adds pressure to reach the goal quickly.
- If the limit is crossed, I subtract 1.0 reward to punish delay.

Dynamic Step Penalty

- If enabled, each extra step makes the penalty slightly worse.
- Formula: -0.04 - 0.001 * steps
  This biases the agent toward shorter, more efficient paths.

Trap Refreshing

- Traps can reset every few episodes (trap_refresh_interval).
- I sample 3 new trap positions from safe, non-goal, non-start tiles.
- This tests re-learning — the agent can't hardcode its trap avoidance strategy.

Transitions

- I precompute transitions using P[(state_idx, action)] = (next_state_idx, reward)
- If the move is invalid (into wall or outside grid), agent stays in place with a −0.1 penalty.
- If it lands on a goal: +1.0 reward
  If it lands on a trap: adds trap penalty
- Else: base step penalty (−0.04 or dynamic)

Reset + Step

- reset() picks the current goal, refreshes traps if needed, sets position to start.
- step() handles movement, slippage, trap penalty, dynamic reward, goal checks, and episode termination.

Render

- Marks the current grid with:
    - A: agent
    - G: goal
- Prints the grid each step for debugging.

Everything above runs consistently for Q-Learning, DQN, and Double DQN. The environment's randomness, moving parts, and hidden penalties force the agents to balance exploration, speed, and caution.

C. Methods

I trained three types of agents on the same environment: tabular Q-Learning, DQN, and Double DQN. All of them used ε-greedy exploration and were evaluated on the same trap/goal/slip setup to keep things fair.

Q-Learning

- Q-table was a NumPy array of shape (states, actions), initialized to zeros
- Agent chose actions using ε-greedy

```
#selecting action using epsilon greedy
if np.random.rand() < epsilon:
    a = np.random.randint(n_actions)
else:
    a = np.argmax(Q[s])

ns, r, done, _ = env.step(a)
```

- This helped balance exploration (especially early on) with exploitation once learning stabilized

- Update rule used the Bellman equation:

```
#updating q table using bellman equation
Q[s, a] += alpha * (r + gamma * np.max(Q[ns]) - Q[s, a])
s, total_r = ns, total_r + r
```

- Ran for 500 episodes with exponential ε decay (0.995 per episode, minimum 0.01)
    - This schedule helped the agent explore more in early episodes and stabilize later
- Episode was considered successful if total reward > 0 (i.e., it reached the goal)
- Rewards were recorded every episode and smoothed using a 10-episode moving average

DQN

- Neural net had two hidden layers with 128 units each and ReLU activations

```
#defining deep q network architecture
class DQN(nn.Module):
    def __init__(self, state_dim, action_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, 128), nn.ReLU(),    #first hidden layer
            nn.Linear(128, 128),       nn.ReLU(),    #second hidden layer
            nn.Linear(128, action_dim)               #output layer (q-values for each action)
        )
    def forward(self, x):
        return self.net(x)
```

    - I used a relatively small net since the state space (10×10) was manageable — enough capacity to generalize, not so large that it overfit

- Input: one-hot encoding of the state index
    - This kept the input compact and let the network focus on transitions and rewards
- Output: Q-values for the 4 actions (up, down, left, right)
- Used a replay buffer (capacity 5000) to store (s, a, r, ns, done) tuples
    - Sampling mini-batches breaks correlation between experiences and helps with stability
- Loss: MSE between predicted Q-values and target Q-values
- Target Q-values were computed using the *target network*, updated every 100 steps

```
#syncing target network every few steps
if steps_done % target_update == 0:
    target_net.load_state_dict(policy_net.state_dict())
```

Target Q formula: **target = reward + gamma * max_a' Q_target(s')**

- Epsilon decayed using an exponential function of step count

```
#defining epsilon decay function
def get_epsilon(t):
    return eps_end + (eps_start - eps_end) * np.exp(-1.0 * t / eps_decay)
```

    - This helped finer control over exploration decay: especially useful when environment had slip
- Trained for 300 episodes, tracked per-episode rewards

Double DQN

- Same model architecture and buffer settings as DQN
- Key difference: action selection and value estimation are split:

```
with torch.no_grad():
    next_actions = policy_net(next_batch).argmax(1, keepdim=True)
    next_q = target_net(next_batch).gather(1, next_actions).squeeze()
    target_q = reward_batch + gamma * next_q * (1 - done_batch)
```

- Policy net selects the best next action
    - Target net evaluates that action's value
- This fixes the known overestimation bias in DQN, especially in stochastic environments
    - Slip and traps in our setup made this issue worse: Double DQN handled it better
- Training loop, epsilon decay, and reward logging matched DQN exactly for fair comparison

Every agent was trained on the same Gridworld: same start point, same trap positions, same goal switching schedule, and same slip probability. I recorded episode rewards and success rates, and saved all curves to outputs/ for later comparison.

D. Experimentation

All agents were trained on the same environment setup. Same grid layout, trap positions, slip probability, moving goal schedule, and episode timeout. I kept everything else consistent so the only thing changing across runs was the algorithm or hyperparameters.

For every run, I tracked:

- total reward per episode
- binary success (1 if reward > 0, else 0)
- smoothed plots using a 10-episode moving average

All results were saved as .npy files to outputs/, along with plotted .png graphs.

Q-Learning

- Ran a single baseline configuration with:
  - $\alpha = 0.1$, $\gamma = 0.99$, $\varepsilon = 1.0$ with decay
  - 500 episodes
- This was my reference for learning without generalization or batching
- Logged reward and success at each episode

DQN

I tested how learning was affected by three things:

- **Replay buffer size**
  - Compared buffer of 5000 (baseline) vs 1000 (small)
  - **Hypothesis:** smaller buffer → less diversity → overfitting or instability

- **Target network update frequency**
  - Compared slow sync (every 100 steps) vs fast (every 20)
  - Faster sync = potentially more responsive but also more volatile

- **Slip**
  - Ran with no slip (0.0) vs high slip (0.2)
  - Tested how sensitive the model was to environment stochasticity

All runs had:

- 300 episodes
- Batch size = 64
- ε-decay based on number of steps
- Same traps, goal config, and penalties

Double DQN

Same experiments as DQN:

- Buffer: 5000 vs 1000
- Target sync: 100 vs 20
- Slip: 0.0 vs 0.2

Only difference was the Q-value update rule. Otherwise, everything (model arch, decay, logging) was held constant to keep the comparison clean.

All experiments were executed through q_learning.py, dqn_training.py, and training_double_dqn.py. I used compare_final.py to generate final plots from the saved .npy files for consistent, centralized comparison.

E. Final Results

I evaluated models using two metrics:

- **Total reward per episode** — tracks how well the agent learns to reach the goal and avoid penalties
- **Success rate** — binary value per episode (1 if reward > 0), smoothed using a 10-episode moving average

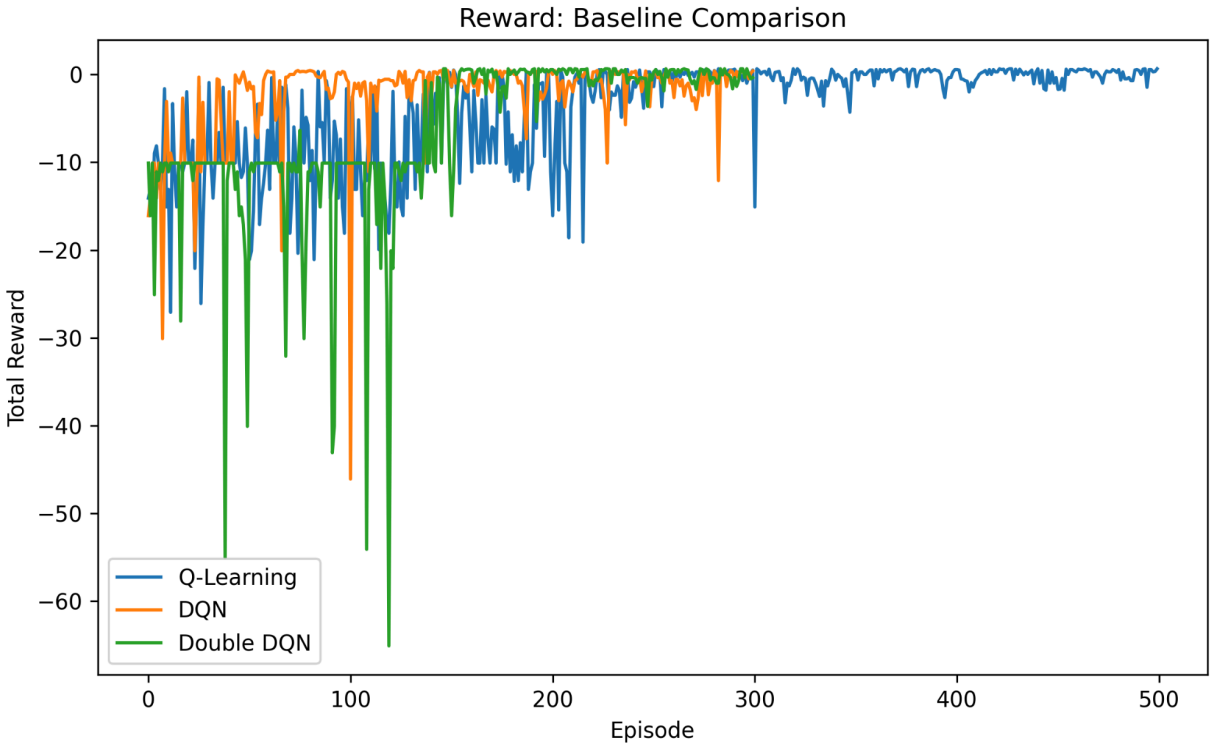
Plots were generated from saved .npy files and include baseline runs and targeted experiments on slippage, buffer size, and target sync rate.

1. Baseline Performance (Q-Learning vs DQN vs Double DQN)

- **Setup**: All models trained with default environment settings (slip = 0.1, traps fixed, buffer = 5000, sync = 100)

Reward:

- Q-learning is really unstable at the start. The reward jumps up and down and takes a long time to improve.
- DQN learns quickly. It finds good rewards early and stays mostly steady after around 100 episodes.
- Double DQN is slower in the beginning and has a few sharp drops, but later on it's the most stable.

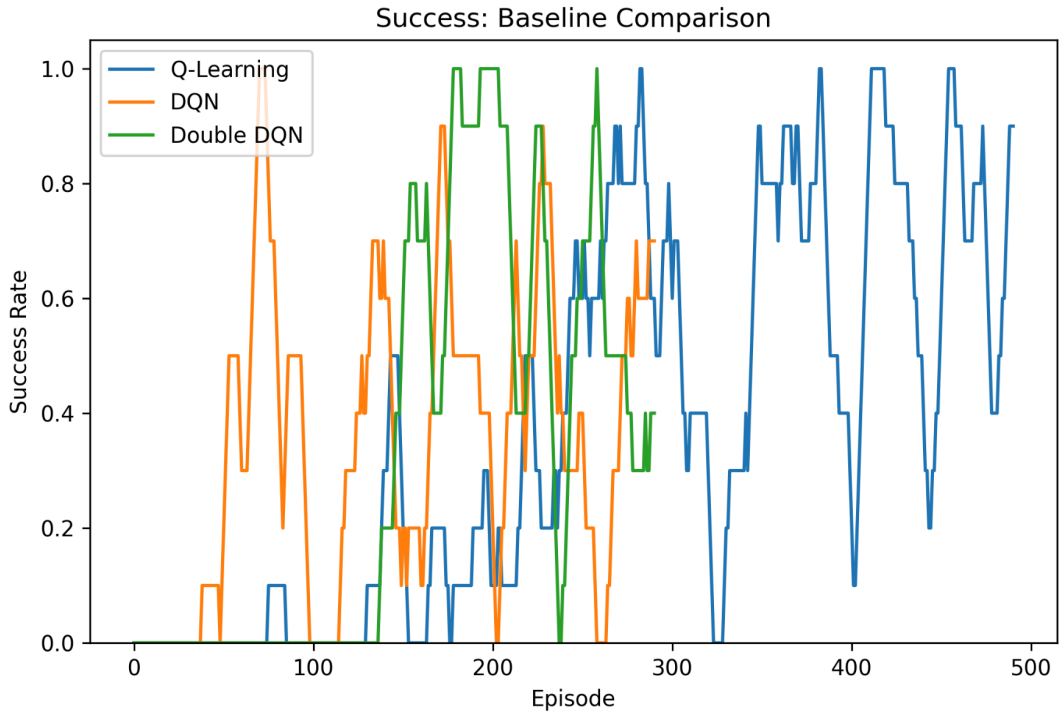


Insights:

- Q-learning struggles because it just memorizes and can't deal with changes like traps or goal movement.
- DQN is fast, but sometimes trusts its own guesses too much, which causes sudden drops.
- Double DQN takes its time and avoids overestimating, so it ends up learning more carefully, but also slowly.

Success rate:

- Q-learning starts quite slow. It takes over 200 episodes to even cross 0.5 success rate, and it stays erratic after that.
- DQN picks up fast, gets good success rates early, but has random drops even later.
- Double DQN is smoother once it starts rising, and holds steady between 0.8 and 1 for a stretch before dipping.

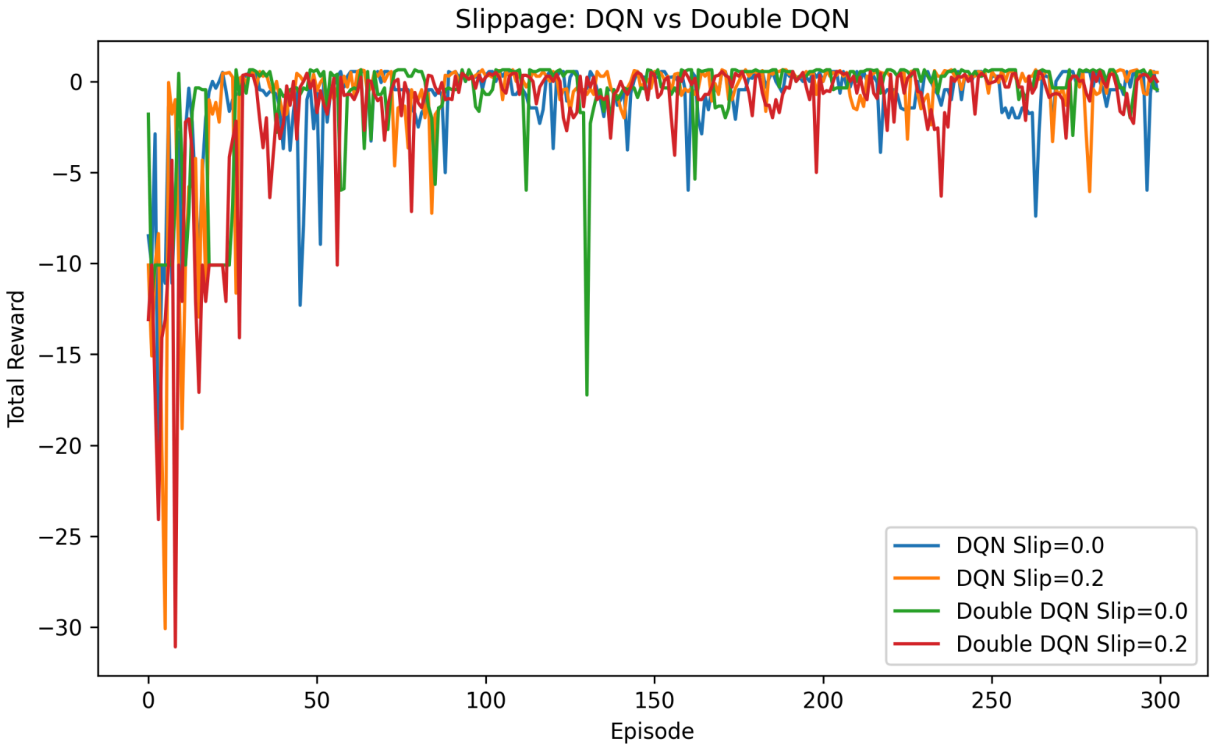


Insights:

- Q-learning needs a ton of time to figure out anything useful, and every change in the environment resets its learning.
- DQN goes for high-reward paths early, but because it overestimates, it sometimes chooses wrongly, which causes drops.
- Double DQN is slower to trust rewards, but once it finds good paths, it sticks to them more reliably.

Slippage Test

- **Setup**: Compared slip = 0.0 vs slip = 0.2 for both DQN and Double DQN
- Without slip, both DQN and Double DQN learn quickly and stay high.
- With slip, DQN shows more drops and takes longer to recover.
- Double DQN with slip also dips sometimes but stays steadier overall

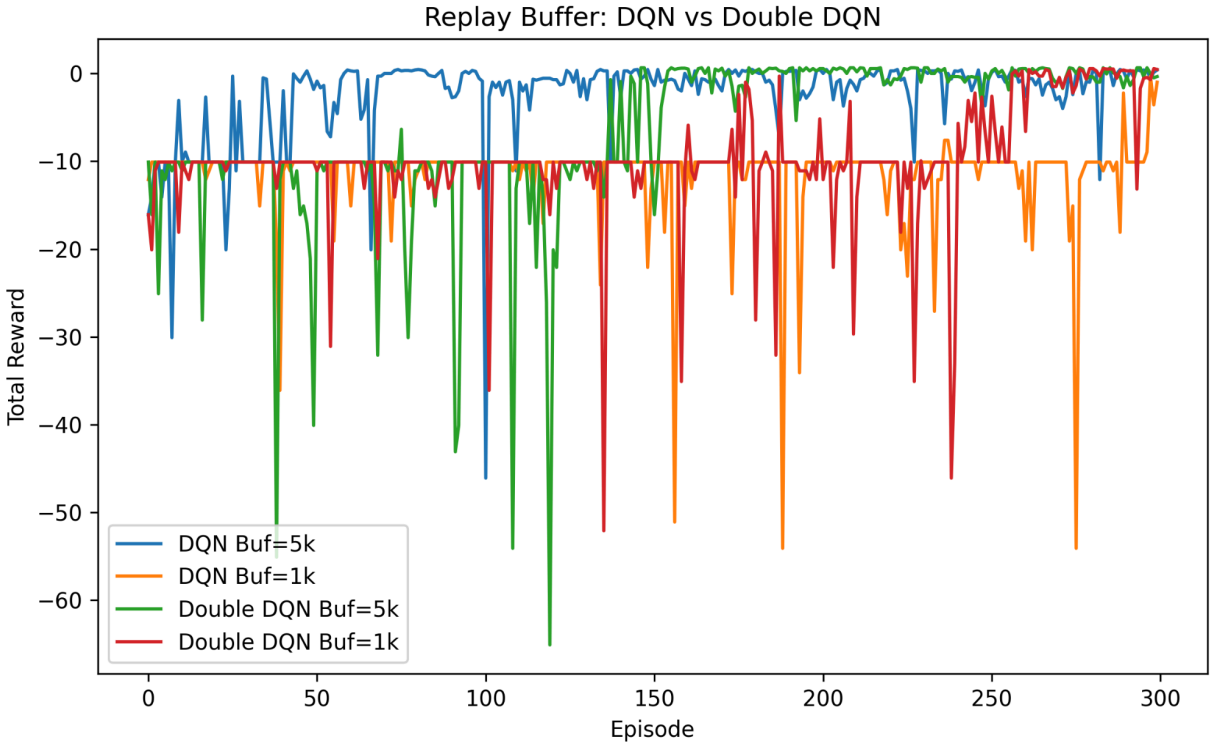


Insights:

Slippage means actions don't always go as planned. DQN gets influenced by lucky outcomes and follows them too strongly. Double DQN is more careful, so it doesn't get misled as often when things go wrong.

3. Replay Buffer Size

- **Setup**: Compared buffer size = 5000 (baseline) vs 1000 (small). Everything else held constant.
    - DQN with a large buffer (5k) does best. It climbs fast and stays high.
    - DQN with a small buffer (1k) is all over the place. Drops are huge and frequent.
    - Double DQN also struggles with a small buffer, but recovers better than DQN.

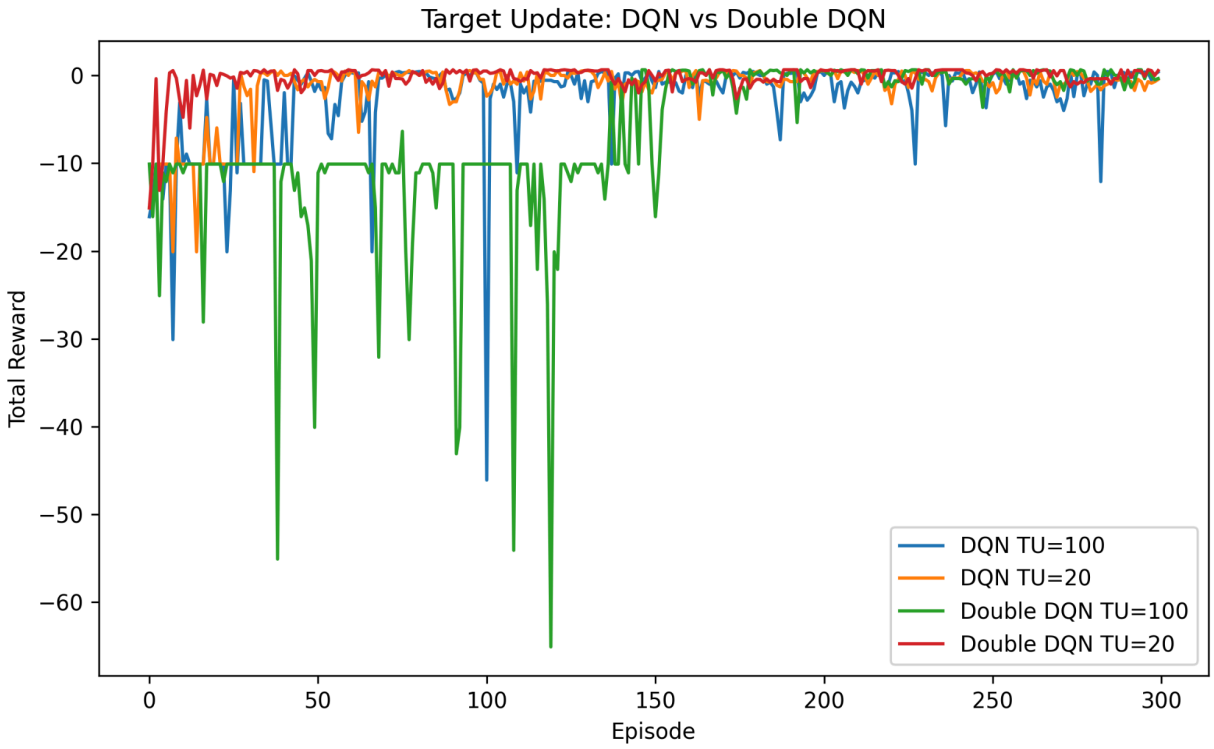


Insights:

- A big buffer means more variety, so the agent learns from different situations.
- A small buffer repeats the same mistakes, so DQN gets stuck or confused.
- Double DQN holds up a bit better because it doesn't overtrust short-term rewards.

4. Target Network Sync Frequency

- **Setup**: Compared sync every 100 steps (baseline) vs every 20 steps (fast)
- DQN with slow updates (TU=100) is steady but has dips.
- DQN with fast updates (TU=20) jumps around a lot but improves faster early on.
- Double DQN with slow updates (TU=100) starts rough, dips hard.
- Double DQN with fast updates (TU=20) is the smoothest and climbs quickest.



Insights:

- When updates happen too fast, it gets unstable since the targets keep changing.
- DQN reacts more to this and ends up learning from wrong updates.
- Double DQN works better with fast updates because it picks and evaluates actions separately, so the targets stay more steady.
- But if updates are too slow, Double DQN keeps using old targets that don't fit what it's doing now, so learning gets erratic and slows down.

F. Conclusion

1. Q-learning behaves the worst out of all three. It's slow at learning, messes up with traps and slip. Better with a more consistent grid.
2. DQN picks up fast and does well in stable setups. Results are quick. But it crashes when there's too much noise or not enough memory.
3. Double DQN handles randomness better. Doesn't respond as adversely to a dynamic environment as the others.
4. Small buffers are bad for both DQN and Double DQN but the latter recovers better.
5. Fast target updates help Double DQN learn faster. DQN can't handle it and ends up unstable.

To conclude, if the environment keeps changing, one should go with Double DQN, with a big buffer, and fast target sync.